

claude-wsl-ubuntu / 4f364eb7-a600-4a2c-9b37-e5a313b6db9d

Metadata

```
- Source: `claude-wsl-ubuntu`  
- Kind: `claude`  
- Source file: `\\wsl.localhost\Ubuntu\home\avidullu\.claude\projects\home-avidullu-projects-khelsutra-guru\4f364eb7-a600-4a2c-9b37-e5a313b6db9d\subagents\workflows\wf_fa7a2077-3a9\agent-a6164396ee6166ae2.jsonl`  
- SHA-256: `c56867f574982284129fd17afcd4f216f6d2b5141763762fabbbab12735315bb7`  
- Source modified: `2026-06-29T10:50:43+00:00`  
- Imported at: `2026-07-05T16:48:30+00:00`  
- project: `wf_fa7a2077-3a9`  
- session_id: `4f364eb7-a600-4a2c-9b37-e5a313b6db9d`
```

Transcript

1. user (2026-06-29T10:50:03.885Z)

Read /home/avidullu/projects/khelsutra-guru/badminton-highlight-indexer/docs/RALLY_DETECTION_CODE_REVIEW.md IN FULL. Summarize findings A, B, and C. Focus on finding B (the rally-end / inactivity temporal-density issue): what was wrong, what the fix changes, why it is a no-op under dense tracking, and any stated acceptance/measurement criteria. Return data, not prose.

2. assistant (2026-06-29T10:50:05.513Z)

I'll read the code review document in full.

3. user (2026-06-29T10:50:06.702Z)

```
1      # Rally Detection Code Review  
2  
3      > **2026-06-29 resolution note.** Re-verified against current `master`: all three  
findings below  
4      > (A `_spread`, B `_trim_inactive_tail`, C `load_canonical_csv`) were still present.  
**A** is fixed  
5      > unconditionally (behaviour-neutral) and **B** is fixed behind a **default-OFF**  
6      > `inactivity_temporal_density` flag ? both in **PR #393** (this repo). B is flag-gated  
because R4 is  
7      > **default-ON in production**, so its fix is a real behaviour change on sparse footage  
that must be  
8      > measured on the golden set before promotion (P2). **C** is tracked for a separate PR  
in the sibling  
9      > `sports-data-collector` repo. See `docs/RALLY_DETECTION_FIXES_PLAN.md` §7/§8 for  
status + the  
10     > promotion gate. The analysis below is preserved as the original review snapshot  
(authored 2026-06-19).  
11  
12     ## 1. How Rally Detection Works  
13  
14     The project identifies rallies through a combination of local trajectory tracking (e.g.,  
TrackNet or WASB) and a series of heuristic gating/trimming rules, culminating in an AI-based  
refinement (Phase 3). Here's a breakdown of the pipeline:  
15  
16     ### Phase 2: Candidate Windowing (The Heuristics)  
17     1. **Velocity Thresholding:** The trajectory detector outputs `(Frame, Visibility, X,  
Y)`. For each visible frame, the normalized velocity (distance divided by frame width per  
second) is calculated. Frames with a velocity exceeding `velocity_thresh` (e.g., 0.02) are  
considered "active" (meaning the shuttle is in flight).  
18     2. **Merging / Grouping:** Active frames within a certain dead time (`merge_gap`,  
usually 2.0s) are merged into candidate windows.  
19     3. **Splitting Over-merged Windows (W1):** If a window exceeds `max_window_duration`,
```

the code splits it at the largest internal gap (dead-time) that is at least ``min_split_gap``. If the window lacks gaps (the shuttle never stops moving), a ``window_hard_cap`` setting (if enabled) forcibly chops it in half at the temporal midpoint.

20 4. ****Inactive Tail Trimming (R4):**** Even after a rally finishes, a player might pick up the shuttle and toss it, creating a slow-moving trajectory tail. To prevent the window from overextending, the algorithm trims the end back to the last segment of the window that had a dense cluster of "fast" frames (velocity > ``rally_thresh``).

21 5. ****Blob Fallback (R5):**** If a window still vastly exceeds the cap after the W1 split and R4 trim (meaning it's an unusually continuous blob of motion), it is chopped down to the maximum cap limit and marked with a ``forced_cut`` flag. This signals that the clip needs "rally-state" cues (like net-crossings) rather than a simple duration cap.

22 6. ****Rally Gate (Gate A):**** In ``rally_gate.py``, a heuristic filters out "false" rallies (like a player tossing a shuttle back to their opponent). It estimates the net position (median play axis coordinate) and counts how many times the shuttle crosses the net. A valid rally must have ``min_crossings`` (e.g., 2).

23

24 **### Phase 3: AI Handoff**

25 Valid candidate windows are padded (e.g., by 2 seconds on each end) and sent to a Generative AI Provider (like Gemini) along with a sports-specific prompt. The AI confirms the semantic boundaries, ensuring the window correctly matches a true rally, and also attempts to count the shuttle exchanges.

26

27 ---

28

29 **## 2. Deep Code Review & Potential Issues Found**

30

31 I performed a deep review of ``tracknet_runner.py``, ``trajectory_hybrid.py``, ``rally_gate.py``, and the parsing logic in ``sports-data-collector``. Here are the findings and hidden issues:

32

33 **### A. Subtle Calculation Artifact in ``_spread`` (``rally_gate.py``)**

34 ****Observation:**** The ``_spread`` function calculates a robust spread (ignoring the bottom 5% and top 5% of coordinates to eliminate outliers):

35 ``lo = s[max(0, int(0.05 * (len(s) - 1)))]``

36 ``hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]``

37

38 ****Hidden Behavior:**** Because ``int()`` truncates rather than rounds, for very short arrays (e.g., ``< 20`` items), the 95th percentile will simply be the second-to-last item, and for ``len(s) == 2``, the spread evaluates to strictly ``0.0``.

39

40 ****Impact:**** This doesn't crash the pipeline, because the play axis is correctly estimated over the *entire* video's thousands of trajectory points. However, if this utility function is reused on small window slices in the future, it will quietly collapse the spread to zero.

41

42 **### B. Sensitivity to Sparse Tracking in ``_trim_inactive_tail`` (``tracknet_runner.py``)**

43 ****Observation:**** The inactivity trimmer identifies the end of a rally by looking for the last frame where the trailing ``trail_frames`` window has a sufficient density of fast-moving frames (``fast_sum / win_count >= min_density``).

44

45 ****Hidden Issue:**** ``win_count`` represents the number of *visible, active frames* in that time boundary, rather than the total elapsed time. If the detector completely loses track of the shuttle for 1.8 seconds and only picks up 1 fast frame right at the end of the window, ``win_count`` will be very low (e.g., 1 or 2). Consequently, ``fast_sum / win_count`` could easily evaluate to 1.0 (100% density).

46

47 ****Impact:**** Sparse tracking points at the very end of a clip might prevent the tail trimmer from cutting off dead-time correctly, artificially inflating the end of the candidate window. This is somewhat mitigated by the ``merge_gap`` breaking trajectories, but it remains a boundary logic quirk.

48

49 **### C. Graceful Degradation on Incomplete CSV Validation (``rally_cvat.py`` in ``sports-data-collector``)**

50 ****Observation:**** The ``parse_cvat_boxes`` function searches zip files for an ``xml`` ending in ``annotations.xml`` or, failing that, *any* ``xml``.

```

51
52     **Issue:** In `load_canonical_csv()`, `safe_int()` is used to catch malformed
`rally_number` fields. If a row is entirely malformed (but doesn't break the CSV parsing), it
just `continue`s silently. While this provides stability, a severely malformed CSV might
silently drop half its rallies without failing the import gate. Logging a warning for skipped
rows would improve observability.
53
54     ### D. Safe Slicing in the W1/R5 Blob Splitters
55     The W1/R5 gap-splitters use a recursive worklist:
56     `split_i = TrackNetRunner._midpoint_split_index(h)`
57     `work.append(h[:split_i]); work.append(h[split_i:])`
58
59     Because it explicitly enforces `len(h) < 2` before splitting, `split_i` will always be
at least 1, meaning the code safely guarantees it will never enter an infinite loop of empty
arrays. This is an excellent, defensively written chunk of code.
60

```

4. assistant (2026-06-29T10:50:09.448Z)

I have read the document in full. Let me also check the referenced fixes plan to capture the stated acceptance/measurement criteria for finding B, since the review document points to it for status and the promotion gate.

5. user (2026-06-29T10:50:11.148Z)

```

-rw-r--r-- 1 avidullu avidullu 8032 Jun 29 10:44 /home/avidullu/projects/khelsutra-
guru/badminton-highlight-indexer/docs/RALLY_DETECTION_FIXES_PLAN.md
---EXISTS---

```

6. user (2026-06-29T10:50:14.319Z)

```

1      <!--
2      PROJECT DOC TEMPLATE ? copy this file to docs/<YOUR_PLAN>.md and fill it in.
3      -->
4
5      # Refactor Rally Detection Edge Cases & Parser Stability
6
7      > **Status:** `IN PROGRESS` . **Owner:** `Agent` . **Created:** `2026-06-19` . **Last
updated:** `2026-06-29`
8      > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived`
9      > **Tracking anchors:** §7 progress tracker is the source of truth; indexed in
`docs/README.md`.
10     > **Relation to existing docs:** Extends `RALLY_DETECTION_QUALITY_REPORT.md`.
11     >
12     > **2026-06-29 update:** the three findings were re-verified against current `master`
(post the
13     > 38-commit advance that touched both `rally_gate.py` and `tracknet_runner.py`) ? **all
three were
14     > still present and unfixed.** P0 (the two in-repo edge cases) is **implemented + unit-
tested in this
15     > same PR (#393)**: the `_spread` fix (A) is behaviour-neutral and lands
unconditionally; the
16     > `_trim_inactive_tail` density fix (B) is a behaviour change on sparse footage (R4 is
**default-ON**
17     > in prod) so it lands behind a **default-OFF** `inactivity_temporal_density` flag,
promoted only
18     > after a golden-set measurement (§8 Q2). P1 lives in the sibling `sports-data-
collector` repo, so it
19     > ships as a **separate PR there** (cannot be in #393).
20
21     ---
22
23     ## 0. TL;DR
24     The rally detection pipeline is largely robust, but a deep code review highlighted three

```

minor edge cases/bugs: a potential divide-by-zero or zero-spread in `\_spread` for tiny sequences, a density calculation vulnerability for sparse tracks in `\_trim\_inactive\_tail`, and silent skipping of malformed rows in the CVAT parsing script. This plan outlines the fixes for these issues to increase stability.

```
25
26     ## 1. Problem & goal
27     The current implementation of the candidate windowing handles normal inputs extremely
28     well (including handling blobs), but could fail or behave erratically under specific conditions:
29     - Zero-spread calculation: `_spread` in `rally_gate.py` evaluates to `0.0` for
30     arrays of length 2, which could cause a zero deadband if used on short clips.
31     - Sparse trailing frames: `_trim_inactive_tail` calculates frame density based on
32     *visible* frames. A long gap followed by a single active frame gives 100% density, artificially
33     inflating the rally end.
34     - Silent failure on malformed CSV: `load_canonical_csv` in `rally_cvat.py` silently
35     ignores rows without a valid `rally_number`.
36
37     Goal: Fix these three issues to harden the rally detection and ingestion pipelines
38     without changing the existing heuristic thresholds.
39
40     ## 4. Design / architecture
41     The changes are entirely localized to the identified functions:
42     - `rally_gate.py` ? done (#393): `_spread` now rounds the p95 index UP with
43     `math.ceil()`, so a
44     2-element (or any short) input falls back to the full min..max range instead of
45     collapsing to
46     `0.0`. On large arrays ceil-vs-truncate differ by at most one sample, so the robust
47     p5/p95 trim is
48     unchanged in practice.
49     - `tracknet_runner.py` ? done (#393), flag-gated default-OFF: `_trim_inactive_tail`
50     gains an
51     `temporal_density` flag. When ON, the density denominator is the trailing window's
52     temporal
53     length** (`min(trail_frames, frames[i] - frames[0] + 1)`) instead of `win_count` (the
54     count of
55     active *samples*, which sparse tracking inflates). Because `group` holds only active
56     frames, with
57     dense tracking the two denominators are identical ? so flipping the flag is a *no-op
58     on
59     well-tracked clips** and changes only the sparse-tracking case. Correction
60     (2026-06-29): R4 is
61     default-ON in production (`window_inactivity_end = 1.5`, promoted at the R1
62     milestone ? an
63     earlier draft of this doc wrongly called it default-OFF), so the fix is a *behaviour
64     change* on
65     sparse footage and ships behind the new `inactivity_temporal_density` flag (default-
66     OFF**, plumbed
67     config ? `trajectory_hybrid` ? eval `WindowingPreset`), to be promoted only after a
68     golden-set
69     measurement at the SERVED operating point ($8 Q2).
70     - `rally_cvat.py` ? separate PR (sibling `sports-data-collector` repo, not this
71     one): introduce a
72     `logger.warning` before each `continue` in `load_canonical_csv` so skipped/malformed
73     rows are
74     surfaced during ingestion.
75
76     ## 6. Honest limits ? what this does NOT do
77     This will NOT change the core AI validation (Phase 3) or the existing `velocity_thresh`
78     values. It solely tightens up boundary calculation edge cases. It will not suddenly "fix"
79     rallies that the tracking model missed entirely due to lighting or occlusion.
80
81     ## 7. Deliverables & progress tracker    ? *source of truth*
82     Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. *One small PR per row*,
83     branched from `origin/master`, no stacking. Update the row (status + PR link) as work lands.
84
85     | ID | Deliverable | Depends on | Gated? | Status | PR |
```

```

62      |----|-----|-----|-----|-----|----|
63      | P0a | Fix `_spread` zero-collapse (behaviour-neutral, unconditional) | ? | No | ? |
#394 |
64      | P0b | `_trim_inactive_tail` temporal density behind `inactivity_temporal_density` flag
(default-OFF) | ? | No | ? | #394 |
65      | P1 | Warn on rows silently skipped by `load_canonical_csv` (sibling `sports-data-
collector`) | ? | No | ? | sdc#50 |
66      | P2a | Expose the flag in the `rally_seg_eval` A/B CLI (`--[vs-]inactivity-temporal-
density`) so the promotion run is one command | P0b | No | ? | this PR |
67      | P2b | **Promote P0b**: golden-set A/B at SERVED windowing ? flip flag default-ON if no
regression | P2a | ? owner | ? | ? |
68
69      ## 8. Open questions ? owner / external
70      1. ~~Do we want `_trim_inactive_tail` to fall back to the existing behavior if there are
very few
71          frames, or strictly enforce a minimum temporal density?~~ **Resolved (#393):** strict
temporal
72          density. Because `group` carries only active frames, the temporal denominator equals
the
73          sample-count denominator under dense tracking, so this is behaviour-preserving for
well-tracked
74          clips and only corrects the sparse case ? no separate few-frames fallback needed.
75      2. **Owner promotion gate (P2b) ? pending.** P0b ships flag-OFF, so the merge is a
76          **zero-regression no-op in production**. The A/B is now a single command (P2a wired
the toggle
77          into `rally_seg_eval`):
78          ...
79          python -m backend.eval.rally_seg_eval \
80              --manifest <golden_manifest.json> --features-dir <golden_features_dir> \
81              --preset served --vs served --vs-inactivity-temporal-density
82          ...
83
84
85          (base = SERVED with the flag OFF, candidate = SERVED with it ON; diffs tolF1 at the
**SERVED**
86          operating point ? R4 is default-ON in prod, so this measures B's real end-boundary
effect on
87          sparse footage.) **Promote** by flipping `IndexingConfig.inactivity_temporal_density`
to `True`
88          (a one-line PR) **only if tolF1 does not regress**. This step is owner-only: it needs
the golden
89          corpus + GPU/WSL-extracted trajectory features, which don't exist offline. Note
finding B is a
90          **no-op under dense tracking**, so the A/B only shows signal on clips where the
shuttle tracker
91          actually drops out ? which is exactly why it can't be measured on synthetic/offline
fixtures.
92
93      ## 9. Definition of done
94      - [x] `_spread` bug fixed and tested (behaviour-neutral, unconditional).
95      - [x] `_trim_inactive_tail` density fix implemented behind `inactivity_temporal_density`
(default-OFF),
96          plumbed config ? segmenter ? eval preset, tested in both flag states.
97      - [ ] **P2 promotion**: golden-set A/B at SERVED windowing, then flip the flag default-
ON if no
98          regression. *(owner-gated, real video ? see §8 Q2)*
99      - [ ] Warning log added for malformed rows in `rally_cvat.py`. *(sibling-repo PR,
separate)*
100     - [x] Tests updated and passing *(offline suite; golden-corpus regression is owner-gated
? see Q2)*.
101
102     ## 10. References
103     **Internal code anchors `[verified]`:**
104     - `backend/pipeline/detectors/rally_gate.py`
105     - `backend/pipeline/detectors/tracknet_runner.py`

```

```
106     - `src/parsers/cvat/rally_cvat.py`  
107     **Internal docs:**  
108     - `docs/RALLY_DETECTION_CODE_REVIEW.md`  
109
```

7. assistant (2026-06-29T10:50:16.587Z)

I have all the information needed from both the code review document and the referenced fixes plan. Let me compile the structured findings.

8. user (2026-06-29T10:50:42.954Z)

Structured output provided successfully